

Name: Gowshick S

Reg No.: 23BCE1200

Project Title:

AeroPassport: Flight Hardware Traceability and Lifecycle Management System

Project Abstract

Aerospace hardware cannot be treated as ordinary inventory. Every critical component must be traceable throughout its lifecycle, including who supplied it, which batch it belongs to, whether it passed inspection, where it was integrated, and what mission events it experienced. This project models that traceability problem in a lightweight and practical way using MongoDB.

AeroPassport stores and manages flight-hardware-related data such as component details, supplier information, batch numbers, certification status, inspection records, mission module assignments, and lifecycle events. Instead of using a frontend interface, the application is implemented as a console-based backend system with API support through Postman or Swagger. The project demonstrates how MongoDB can efficiently handle document-oriented storage, nested and referenced data, and multiple relationship types while fulfilling the certification requirements for connection handling, CRUD operations, insert/update/delete/drop actions, and mapping between entities.

Project Objective

The objective of AeroPassport is to demonstrate effective MongoDB usage through a console-based application.

1. **Database connection and console-based execution** using a backend framework connected to MongoDB.
 2. **API validation through Postman (primary) and Swagger** for testing operations.
 3. **Create operations** for the main entities in the system.
 4. **Drop, update, insert, and delete operations** for collections and records.
 5. **Insert functionality** for adding new records and related nested data.
 6. **Relationship mapping** between entities using one-to-one, one-to-many, many-to-one, and many-to-many models.
-

Technical Description

Technology Stack

Backend Framework

- Java 21
- Spring Boot

Database

- MongoDB

API Documentation and Testing

- Springdoc OpenAPI
- Swagger UI
- Postman

Build and Dependency Management

- Apache Maven
- Maven Wrapper (mvnw)

IDE

- Antigravity IDE
-

Database Design

The application consists of four collections.

1. Suppliers Collection

Stores supplier information.

Purpose:

- Maintain supplier records.
- Track origin of components.

2. Parts Collection

Stores individual flight hardware components.

Purpose:

- Maintain hardware inventory.
- Associate components with suppliers.
- Track operational status.

3. Part Passports Collection

Stores the digital identity of components.

Purpose:

- Maintain traceability.
- Store embedded inspection records.
- Store lifecycle events.
- Maintain readiness status.

4. Modules Collection

Stores subsystem assemblies.

Purpose:

- Group multiple components.
- Track module composition.

MongoDB Data Modeling Strategy

The project uses a hybrid document modeling approach.

→ Referencing

Examples:

Supplier → Parts

Part → Passport

Module ↔ Parts

Benefits:

- Reduced redundancy.
- Easier updates.

→ Embedding

Examples:

Passport

Inspections

Events

Benefits:

- Faster retrieval.
 - Improved traceability.
 - Atomic updates.
-

MongoDB Indexing Strategy

Indexes are implemented to improve query performance.

Collection	Indexed Field	Type
parts	supplierId	Single Field
parts	batchNumber	Single Field
part_passports	partId	Unique
modules	partIds	Multikey

REST API Design

Base URL:

<http://localhost:8080/api>

→ Supplier APIs

Manage supplier records.

Method	Endpoint	Description
POST	/suppliers	Create supplier
GET	/suppliers	Retrieve all suppliers
GET	/suppliers/{id}	Retrieve supplier by ID
PUT	/suppliers/{id}	Update supplier
DELETE	/suppliers/{id}	Delete supplier

→ Part APIs

Manage hardware components.

Method	Endpoint	Description
POST	/parts	Register new part
GET	/parts	Retrieve all parts
GET	/parts/{id}	Retrieve part

PUT	/parts/{id}	Update part
DELETE	/parts/{id}	Delete part

→ Module APIs

Manage subsystem modules.

Method	Endpoint	Description
POST	/modules	Create module
GET	/modules	Retrieve modules
GET	/modules/{id}	Retrieve module
PUT	/modules/{id}	Update module
DELETE	/modules/{id}	Delete module

→ Reporting APIs

Generate MongoDB-powered reports.

Hardware Genealogy Report - GET /api/parts/{partId}/genealogy

Description:

Returns the complete lifecycle and traceability history of a component, including:

- Part information

- Supplier details
- Passport details
- Embedded inspections
- Embedded events
- Current module assignment
- Flight readiness status
- Derived genealogy status

→ Administrative APIs

Database Statistics

GET /api/admin/statistics

Returns:

- Total suppliers
- Total parts
- Flight-ready parts
- Parts under inspection
- Total modules

→ Database Reset

DELETE /api/admin/reset

Purpose:

Drops application collections for testing and demonstration purposes.

Logging Strategy

Spring Boot logging is used to record major lifecycle actions.

Examples:

INFO Supplier created: SUP001

INFO Part registered: PAR001

INFO Passport generated: PAS001

INFO Event appended: Received

INFO Genealogy report generated.

Purpose:

- Console-based execution
 - Improve auditability.
-

Verification and Testing Strategy

→ Swagger UI

Used for:

- Endpoint discovery.
- Interactive testing.

→ Postman

Used for:

- CRUD verification.
- Report validation.
- Administrative operation testing.

→ MongoDB Compass

Used for:

- Document inspection.
-